



Observações Gerais

- É vedado o uso de ferramentas de inteligência artificial ou modelos de linguagem (LLMs).
- A constatação de utilização de tais recursos implicará na atribuição de nota **zero**.
- O discente poderá ser chamado para apresentar e justificar o código desenvolvido.
- A reprodução parcial ou integral do trabalho de outro colega resultará igualmente em nota **zero**.
- Crie um repositório no GitHub, GitLab ou Bitbucket para o envio da atividade no SIGAA.
- Durante a aula, algumas resoluções dos alunos serão selecionadas por sorteio para debate.

Análise inicial de complexidade

Avaliar o desempenho dos algoritmos de busca linear e busca binária em diferentes cenários, medindo:

1. Número de comparações realizadas.
2. Tempo de execução do algoritmo.
3. Diferença de desempenho ao buscar elementos no início, meio e fim do vetor.

Regras da atividade

1. Criar vetores de diferentes tamanhos:
 - 1.000 elementos
 - 10.000 elementos
 - 100.000 elementos
2. Preencher os vetores com números inteiros aleatórios já ordenados.
3. Implementar duas funções de busca:
 - **buscaLinear(int[] vetor, int valor)**
 - **buscaBinaria(int[] vetor, int valor)**
4. Para cada vetor, realizar buscas em três posições:
 - Valor no início (primeiro elemento).
 - Valor no meio (elemento central).
 - Valor no fim (último elemento).
5. Medir
 - Comparações: adicionar um contador dentro das funções para contar quantas vezes uma comparação foi feita.
 - Tempo de execução: usar a classe `System.nanoTime()` para capturar o tempo antes e depois da execução.

```

long inicio = System.nanoTime();

// Algoritmo aqui

long fim = System.nanoTime();
long duracaoMs = (fim - inicio) / 1000000;
System.out.println("Tempo: " + duracaoMs + " ms");

```

6. Escreva uma discussão sobre os resultados obtidos, destacando os cenários de **melhor caso**, **caso médio** e **pior caso** para ambos os algoritmos.

Análise Final

- Além da busca binária implementada manualmente, teste o método `Arrays.binarySearch`:

```

import java.util.Arrays;

long inicio = System.nanoTime();
int pos = Arrays.binarySearch(vetor, valor);
long fim = System.nanoTime();

long duracaoMs = (fim - inicio) / 1000000;
System.out.println("Tempo Arrays.binarySearch: " + duracaoMs + " ms");

```

Avalie se há diferença significativa de tempo entre a implementação própria e a versão otimizada da biblioteca Java.

- Suponha que você esteja procurando uma palavra em um dicionário. O dicionário tem 240000 palavras. Na **pior** das hipóteses, de quantas etapas você acha que a busca sequencial e busca binária precisaria? Consegue generalizar para qualquer entrada n ?

Desafio

Além da **busca sequencial** e da **busca binária**, existem outros algoritmos de busca que podem ser aplicados em vetores ordenados. Dois exemplos são:

- Busca por Interpolação:** utiliza uma estimativa baseada na posição relativa do valor procurado. É especialmente eficiente quando os dados estão **uniformemente distribuídos**.
- Busca de Fibonacci:** uma variação da busca binária que utiliza números de Fibonacci para determinar os pontos de divisão, evitando cálculos diretos de médias.

Atividade: Escolha uma das duas alternativas acima (**Interpolação** ou **Fibonacci**), pesquise sua implementação e compare o tempo de execução com a busca binária tradicional.

- Implemente o algoritmo escolhido.
- Teste-o em vetores de diferentes tamanhos (por exemplo, 1.000, 10.000 e 100.000 elementos).
- Compare o tempo de execução com a busca binária, para medir o desempenho.